Web Application Development

- REST, Restfull Api

Ammar Ahmad Khan

What is an API?

API stands for **Application Programming Interface**.

It is a **set of rules and protocols** that allows different software applications to communicate with each other.

Think of an API like a **waiter** at a restaurant:

- You (client) tell the waiter what you want (send request).
- The waiter brings your order from the kitchen (server).
- You don't need to know how the food is made; you just interact with the waiter.

Example:
A weather app uses an API to get real-time weather from a weather data provider.

What is a REST API?

REST stands for **Representational State Transfer**.

A **REST API** is an API that follows the **REST architectural style**, which uses **HTTP methods** to interact with **resources**.

REST APIs are the most common way to build **web services** that can be accessed from browsers, mobile apps, or other systems.

Why is REST API Needed?

System Communication: REST allows different systems (mobile app, website, backend server) to talk over HTTP.

Platform Independence: Frontend and backend can be built in different technologies.

Reuse: A single REST API can be used by web apps, mobile apps, and third-party developers.

Scalable: REST is stateless, so servers handle many clients efficiently.

Standardization: Uses common HTTP methods (GET, POST, PUT, DELETE) and standard status codes (200, 404, 500).

REST Architecture Principles Explained

1. Resource-Based Design

- In REST, everything is treated as a **resource**.
- A resource could be a **user, product, book, order**, etc.
- Each resource is identified using a **URI** (Uniform Resource Identifier).

Example:
/users/10 refers to the **user** resource with ID 10.

REST Architecture Principles Explained

2. URI Identification

- URIs should use **nouns**, not **verbs**.
- REST focuses on **what** the resource is, not **how** you act on it.

Good:

GET /products/5

Bad:

GET /getProductById?id=5

REST Architecture Principles Explained

3. HTTP Method Selection

REST uses standard HTTP methods to perform actions on resources.

GET-Read-Retrieve data
POST-Create-Add a new resource
PUT-Update-Replace an existing resource
PATCH-Partial Update-Modify part of a resource
DELETE-Remove-Delete a resource

Example:
POST /books → Add a new book
DELETE /books/3 → Delete book with ID 3

REST Architecture Principles Explained

4. Statelessness

- Each client request must contain **all the information** needed by the server to understand and process it.
- **Server does not store any session or user data** between requests.

Benefits:

- Easier to scale (no session memory needed)
- Simpler design

You cannot say “Remember me” you must always send credentials or tokens.

REST Architecture Principles Explained

5. Idempotency

- An **idempotent** operation is one that can be performed **multiple times** and will produce the **same result** every time.

GET-Yes-Fetching data doesn't change it
PUT-Yes-Replaces data, same result each time
DELETE-Yes-Deleting again has no additional effect
POST-No-Creates a new item each time

Example:
Calling DELETE /users/5 3 times will remove user 5 only once.

Building a Simple REST API Example (Node.js + Express)

We will build a small API for managing a book collection.

Setup

```
npm init -y
npm install express
```

index.js file

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.json());

let books = [
  { id: 1, title: 'The Alchemist', author: 'Paulo Coelho' },
  { id: 2, title: '1984', author: 'George Orwell' }
];

// GET all books
app.get('/books', (req, res) => {
  res.json(books);
});

// GET a single book by ID
app.get('/books/:id', (req, res) => {
  const book = books.find(b => b.id == req.params.id);
  book ? res.json(book) : res.status(404).send('Book not found');
});
```

```
// POST a new book
app.post('/books', (req, res) => {
  const newBook = {
    id: books.length + 1,
    title: req.body.title,
    author: req.body.author
  };
  books.push(newBook);
  res.status(201).json(newBook);
});

// PUT (update) a book
app.put('/books/:id', (req, res) => {
  const book = books.find(b => b.id == req.params.id);
  if (book) {
    book.title = req.body.title;
    book.author = req.body.author;
    res.json(book);
  } else {
    res.status(404).send('Book not found');
  }
});

// DELETE a book
app.delete('/books/:id', (req, res) => {
  books = books.filter(b => b.id != req.params.id);
  res.status(204).send();
});

app.listen(port, () => {
  console.log(`REST API running at http://localhost:${port}`);
});
```